FUNDACIÓN DE ESTUDIOS SUPERIORES

FESC

DOSSIER TÉCNICO — ARQUITECTURA Y DISEÑO DE SOFTWARE

Guías Prácticas N°1 al N°4 — Proyecto GoApple

Proyecto	GoApple — Sistema Web de Gestión Comercial y de Inventario
Asignatura	Arquitectura y Diseño de Software
Docente	Ing. Robinson Damián Gómez Sánchez
Estudiantes	James Quintero Mosquera Giorgi Julián Ordóñez
Institución	Fundación de Estudios Superiores FESC
Sede	San José de Cúcuta
Fecha	10 de marzo de 2026

ARQUITECTURA Y DISEÑO DE SOFTWARE
SAN JOSÉ DE CÚCUTA — 2026

GUÍA N°1 DEFINICIÓN DE ARQUITECTURA DE SOFTWARE PARA GOAPPLE

Fecha	10 de febrero de 2026
Autores	James Quintero Mosquera Giorgi Julián Ordóñez

Actividad 1: Planteamiento del Proyecto (Arquitectura de Negocio)

Esta sección define el "qué" y el "por qué" del sistema GoApple.

Título del Proyecto: *GoApple — Sistema Web de Gestión Comercial y de Inventario*

Problema a resolver: Actualmente, la gestión de productos Apple (iPhone, iPad, MacBook, accesorios) se realiza de manera manual o en hojas de cálculo dispersas. Esto genera:

- Descontrol en el inventario.
- Errores en el registro de ventas.
- Falta de seguimiento a clientes.
- Dificultad para generar reportes financieros.
- Riesgo de pérdida de información.

GoApple surge como una solución web centralizada que permitirá administrar productos, clientes, ventas y reportes de manera organizada y segura.

Alcance — 3 procesos clave:

1. Gestión de Inventario

- Registro de productos (nombre, categoría, precio, stock).
- Actualización de existencias.
- Control de entradas y salidas.
- Alertas por bajo stock.

2. Gestión de Ventas

- Registro de ventas.
- Asociación de productos vendidos.
- Cálculo automático de totales.
- Generación de comprobante.

3. Gestión de Clientes

- Registro de clientes.
- Historial de compras.

- Consulta de información de contacto.

Actividad 2: Modelado de la Arquitectura Conceptual

Se define la estructura del sistema siguiendo el patrón de arquitectura en capas:

Capa de Presentación (Frontend)

- Tecnologías: HTML5 (estructura semántica), CSS3 (diseño responsivo), JavaScript (validación de formularios).
- Función: Interfaz con la que interactúan Administrador y Vendedor.

Capa de Negocio (Backend)

- Tecnología: PHP.
- Función: Contiene la lógica del sistema (no permitir venta sin stock, calcular totales, validar clientes, actualizar inventario).

Capa de Datos (Persistencia)

- Tecnología: MySQL.
- Tablas principales: productos, clientes, ventas, detalle_venta, usuarios.

Estructura física de directorios:

```
/goapple
│
├── /assets   → Recursos visuales (Presentación)
│   ├── /css
│   └── /js
│
├── /views    → Vistas (Interfaz)
│   ├── login.php
│   ├── dashboard.php
│   ├── productos.php
│   ├── ventas.php
│   └── clientes.php
│
├── /classes  → Lógica de Negocio (PHP)
│   ├── Usuario.php
│   ├── Producto.php
│   ├── Cliente.php
│   ├── Venta.php
│   └── Detalle_Venta.php
│
├── /config   → Configuración y BD
│   └── database.php
│
├── /includes → Componentes reutilizables
│   ├── header.php
│   └── footer.php
│
└── index.php   → Punto de entrada
```

Actividad 3: Definición de la Arquitectura Física

Se documenta el entorno donde se ejecutará GoApple:

Componente	Especificación Propuesta
Servidor Web	Estándar con PHP/XAMPP
Motor de Base de Datos	Relaciones complejas, integridad referencial
Lenguaje de Scripting	Compatibilidad y seguridad reciente
Navegador Cliente	Chrome / Edge
Sistema Operativo	Windows 10/11

Actividad 4: Investigación — Patrones de Diseño (Refactoring.Guru)

¿Qué es un patrón de diseño? Son soluciones reutilizables y probadas para problemas comunes en el desarrollo de software.

En GoApple se aplicará el patrón arquitectónico MVC (Modelo–Vista–Controlador):

- Modelo → Clases Producto, Cliente, Venta.
- Vista → Archivos en /views.
- Controlador → Lógica que conecta vistas con modelos.

Reusó: Al separar la lógica de inventario en Producto.php, esta podría reutilizarse en una aplicación móvil, una API o un sistema POS físico, sin reescribir la lógica.

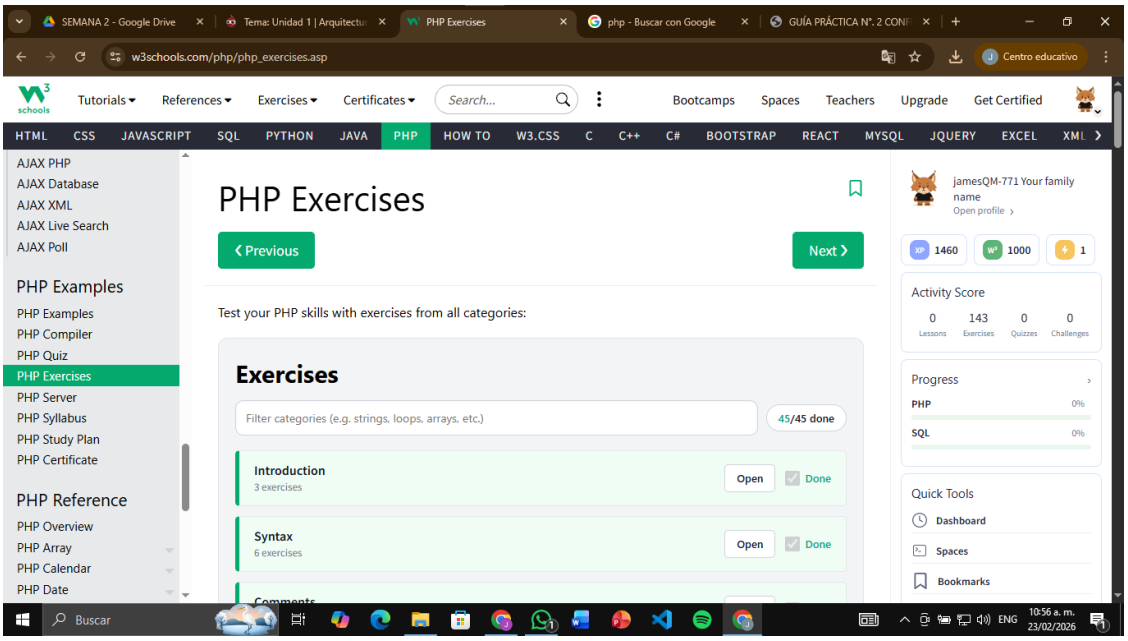
Evolución: Si GoApple integra pasarela de pagos, facturación electrónica o roles avanzados, solo será necesario modificar la capa de negocio.

GUÍA N°2 CONFIGURACIÓN DE ENTORNOS Y CAPAS EN PHP PARA GOAPPLE

Fecha	17 de febrero de 2026
Autores	James Quintero Mosquera Giorgi Julián Ordóñez

Actividad 1: Entrenamiento Intensivo (W3Schools PHP Exercises)

Se ingresó a W3Schools PHP Exercises y se realizaron todos los ejercicios prácticos de la plataforma: Syntax, Variables, Strings, Operators, If...Else, Switch, Loops, Functions, Arrays. A continuación se muestra la evidencia de los ejercicios completados con indicador "Correct":

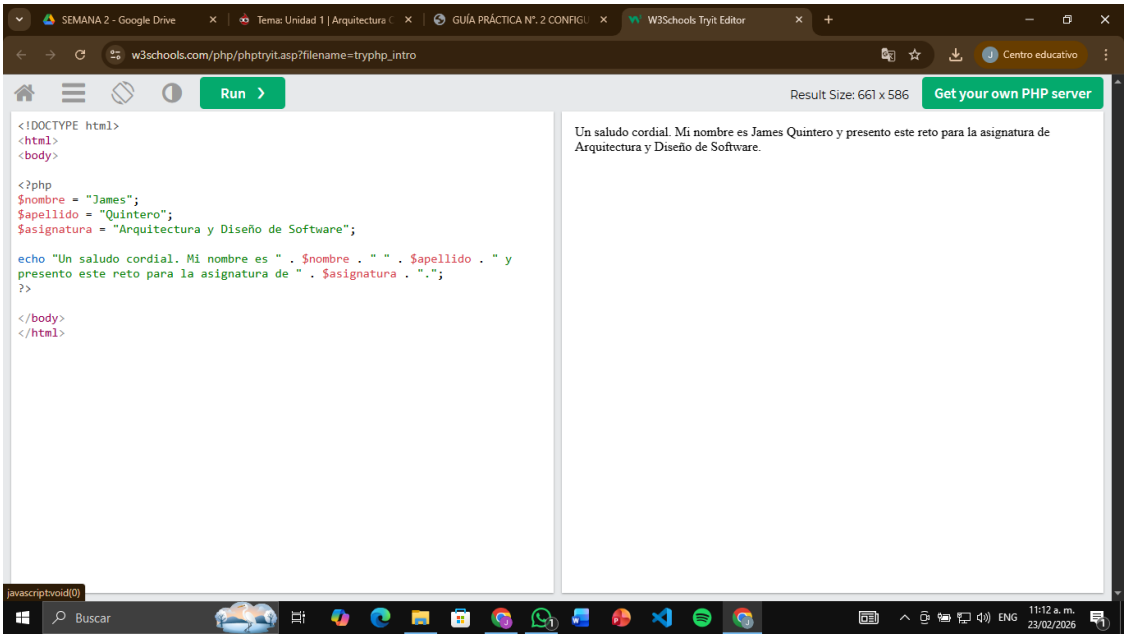


Actividad 2: Retos de Escritura de Código (W3Schools PHP Tryit Editor)

Sin ayudas de "completar", se escribieron desde cero en el Tryit Editor los siguientes retos:

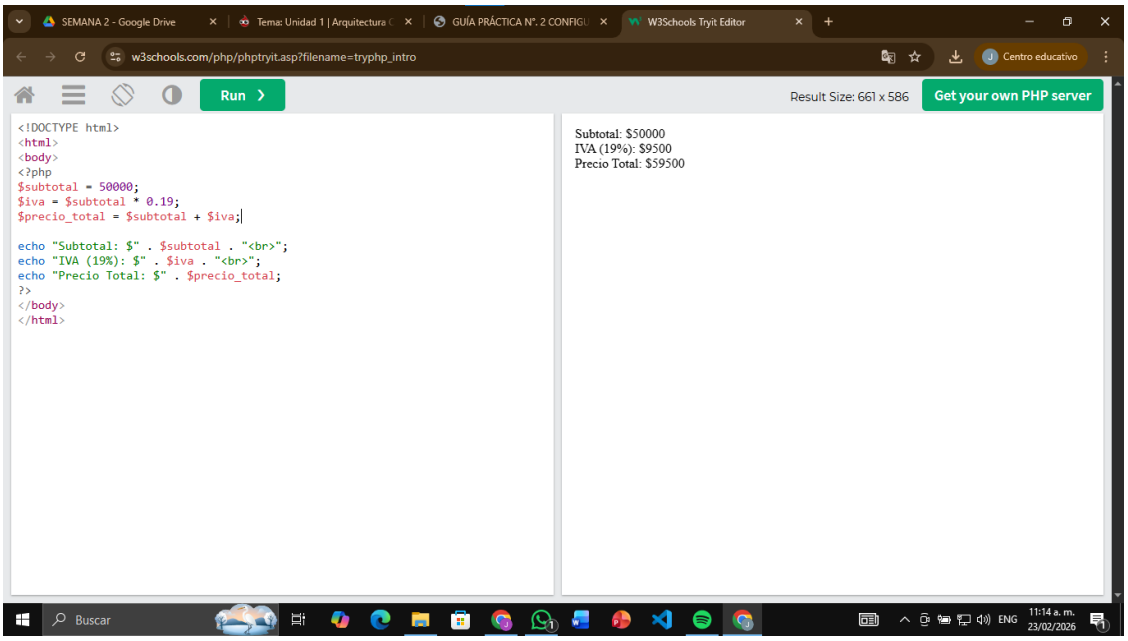
Reto 1 — Variables: Saludo formal con concatenación

Se declararon las variables nombre, apellido y asignatura, y se mostró un saludo formal usando concatenación de cadenas.



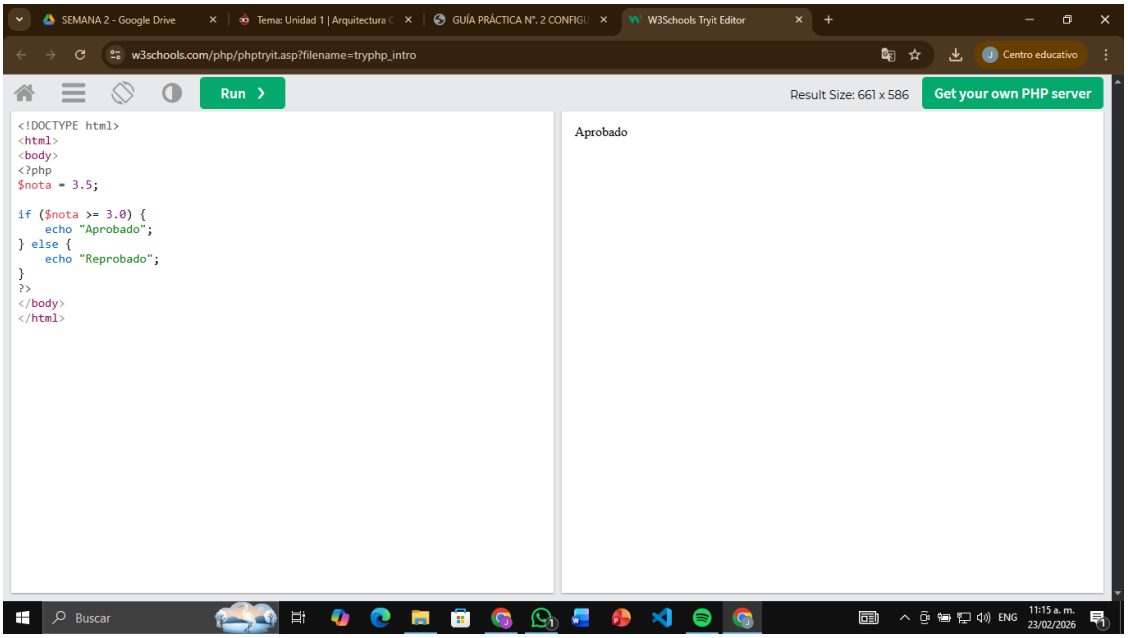
Reto 2 — Operadores: Cálculo de IVA (19%)

Se definió la variable $subtotal, se calculó el IVA al 19% y el precio total, mostrando los tres valores etiquetados.



Reto 3 — Lógica Condicional: Aprobado / Reprobado

Se usó if/else para evaluar la variable $nota. Si es ≥ 3.0 muestra "Aprobado", de lo contrario "Reprobado".



Actividad 3: Investigación Técnica — Frameworks PHP

¿Qué es un Framework PHP?

Un Framework PHP es una estructura de desarrollo que proporciona un conjunto organizado de herramientas, bibliotecas y buenas prácticas para crear aplicaciones web de manera más rápida, segura y estructurada. En lugar de comenzar desde cero, el desarrollador utiliza una base ya construida que incluye:

- Sistema de enrutamiento (manejo de URLs).
- Gestión de bases de datos.
- Plantillas para vistas.
- Manejo de sesiones y autenticación.
- Protección contra vulnerabilidades comunes.
- Organización bajo el patrón MVC.

Laravel — Características Principales

Característica	Descripción	Aplicación en GoApple
Arquitectura MVC	Separación clara de Modelo, Vista y Controlador.	Modelos: Productos, Clientes, Ventas. Vistas: Panel admin. Controladores: Lógica de inventario.
Eloquent ORM	Interacción con BD usando objetos en lugar de SQL manual. Producto::all() en lugar de SELECT * FROM productos.	Simplifica gestión de inventario, reduce errores.
Autenticación Integrada	Registro, login, roles, recuperación de contraseña, rutas protegidas.	Diferencia administrador/vendedor, restringe módulos críticos.

Característica	Descripción	Aplicación en GoApple
Seguridad Integrada	Protección contra SQL Injection, CSRF, XSS. Encriptación de contraseñas.	Protege datos de clientes, ventas y reportes.
Migraciones de BD	Versionamiento de la base de datos.	Control de cambios estructurales en equipo.

Slim — Micro-Framework PHP

Slim es un micro-framework PHP ligero y minimalista, diseñado para crear APIs y aplicaciones pequeñas de manera rápida y eficiente. Sus características son: ligereza (recibir petición → procesar → retornar respuesta), enrutamiento sencillo, modularidad, y alto rendimiento al consumir menos recursos.

Aplicación en GoApple: Slim sería útil si GoApple se enfocara en exponer una API para una app móvil, o si se quisiera separar frontend y backend en un modelo de microservicios.

Criterio de Reusó: Cómo ayuda un Framework

Criterio	Sin Framework	Con Framework
Autenticación	Programar desde cero (horas)	Módulo integrado (minutos)
Seguridad	Implementación manual e inconsistente	Protección automática por defecto
Estructura del código	Variable según el desarrollador	Convención estándar (MVC)
Escalabilidad	Difícil agregar módulos nuevos	Módulos independientes reutilizables

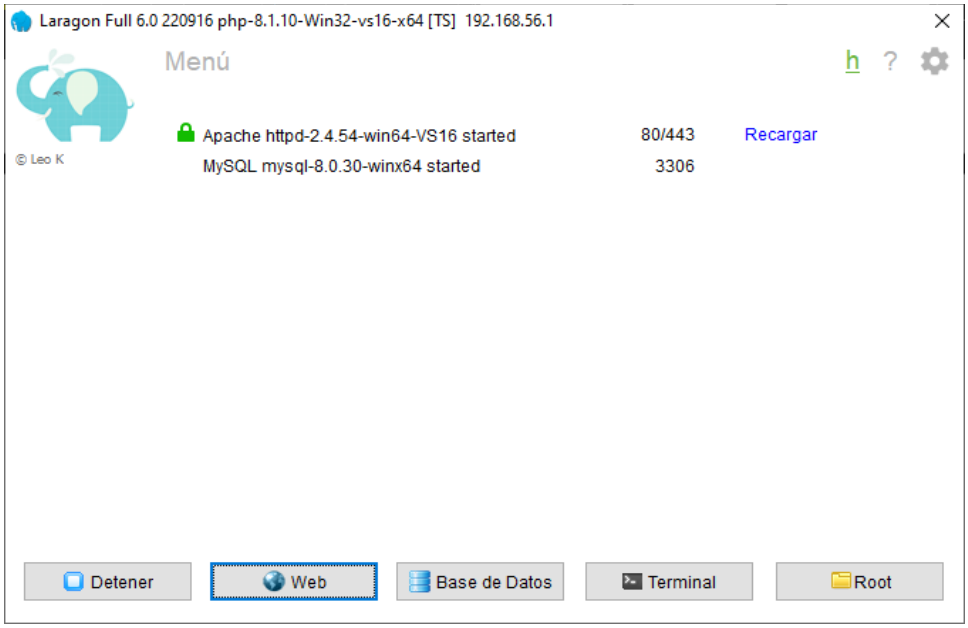
Decisión: PHP Nativo para GoApple

Para el proyecto GoApple, en su fase inicial, se ha decidido utilizar PHP Nativo por las siguientes razones:

- Comprensión profunda de la arquitectura: entender la interacción entre capas e implementar MVC manualmente fortalece el aprendizaje técnico.
- Control total del código: diseñar la arquitectura personalizada sin dependencias externas en fase formativa.
- Proyecto académico/formativo: obliga a implementar validaciones, seguridad y estructura manualmente, comprendiendo qué automatiza un framework.

Actividad 4: Arquitectura de Capas en VS Code / Laragon

Configuración del entorno: Se inició Laragon (Apache/MySQL en verde) y se abrió en VS Code la carpeta del proyecto en C:/laragon/www/.



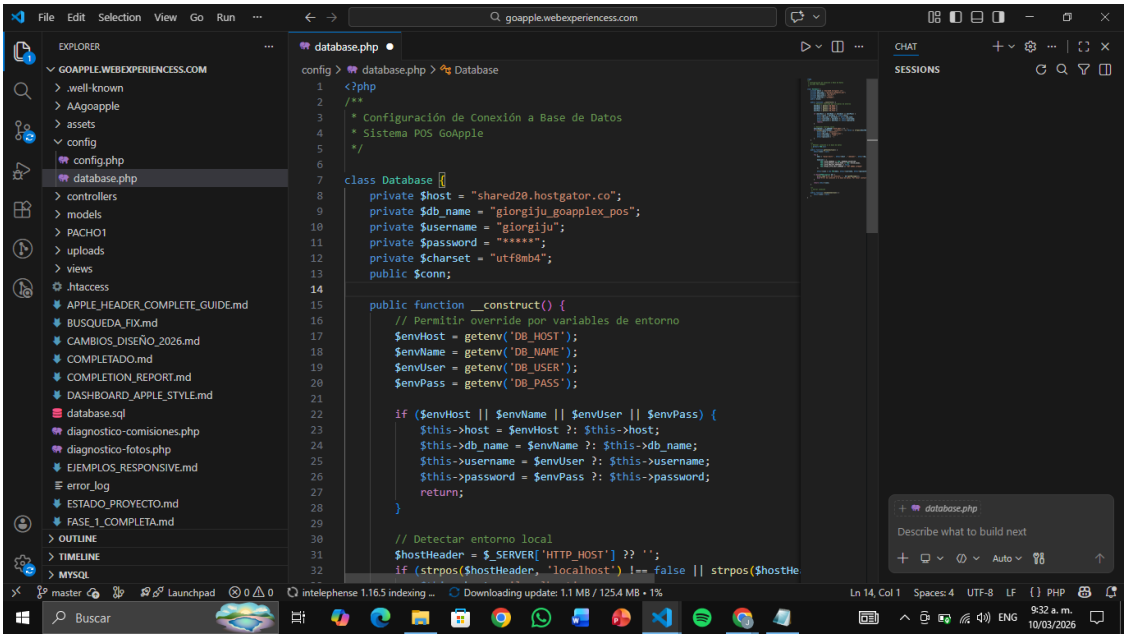
Estructura física creada manualmente con las carpetas que representan la arquitectura de capas: /app (Core y Data), /public (CSS, JS, imágenes e index.php), /views (archivos de interfaz HTML).

Actividad 5: Prueba de Integración — Conexión de Capas

Para verificar que la estructura de carpetas funciona, se realizó el siguiente puente entre archivos:

- Paso A: Se creó conexion.php en /app/Data/ con el código de conexión a la base de datos.
- Paso B: En index.php (raíz) se usó include para llamar al archivo anterior.
- Paso C: Se accedió desde el navegador en localhost/nombre_proyecto y se verificaron ambos mensajes de éxito.

Evidencia — Integración exitosa (VS Code + Navegador):


File Edit Selection View Go Run
goapple.webexperiencess.com
EXPLORER
GOAPPLE.WEBEXPERIENCESS.COM
.well-known
AAgoapple
assets
config
config.php
database.php
controllers
models
PACHO1
uploads
views
.htaccess
APPLE_HEADER_COMPLETE_GUIDE.md
BUSQUEDA_FIX.md
CAMBIOS_DISEÑO_2026.md
COMPLETADO.md
COMPLETION_REPORT.md
DASHBOARD_APPLE_STYLE.md
database.sql
diagnostico-comisiones.php
diagnostico-fotos.php
EJEMPLOS_RESPONSIVE.md
error_log
ESTADO_PROYECTO.md
FASE_1_COMPLETA.md
OUTLINE
TIMELINE
MYSQL
database.php
config > database.php > Database
<?php
/**
* Configuración de Conexión a Base de Datos
* Sistema POS GoApple
*/
class Database {
private $host = "shared20.hostgator.co";
private $db_name = "giorgiju_goapplex_pos";
private $username = "giorgiju";
private $password = "*****";
private $charset = "utf8mb4";
public $conn;
public function __construct() {
// Permitir override por variables de entorno
$envHost = getenv('DB_HOST');
$envName = getenv('DB_NAME');
$envUser = getenv('DB_USER');
$envPass = getenv('DB_PASS');
if ($envHost || $envName || $envUser || $envPass) {
$this->host = $envHost ?: $this->host;
$this->db_name = $envName ?: $this->db_name;
$this->username = $envUser ?: $this->username;
$this->password = $envPass ?: $this->password;
return;
}
// Detectar entorno local
$hostHeader = $_SERVER['HTTP_HOST'] ?? '';
if (strpos($hostHeader, 'localhost') !== false || strpos($hostHe
CHAT
SESSIONS
database.php
Describe what to build next
Auto
master
Launchpad
intelephense 1.16.5 indexing ...
Downloading update: 1.1 MB / 125.4 MB • 1%
Ln 14, Col 1
Spaces: 4
UTF-8
LF
{} PHP
Buscar
ENG
9:32 a. m.
10/03/2026

GUÍA N°3 DIAGRAMACIÓN DE COMUNICACIÓN Y SISTEMAS EXTERNOS PARA GOAPPLE

Fecha	24 de febrero de 2026
Autores	James Quintero Mosquera Giorgi Julián Ordóñez

Actividad 1: Análisis de Interoperabilidad y Estándares de Datos

1. Definición técnica de una API y su rol estratégico

Una API (Application Programming Interface) es un conjunto de reglas y protocolos que permite que diferentes aplicaciones de software se comuniquen entre sí. En el contexto de GoApple POS, la API actúa como el puente mediador: permite que la capa de presentación (navegador web del cajero) consuma los servicios de la lógica de negocio (servidor PHP) sin conocer su implementación interna, asegurando bajo acoplamiento y facilitando la escalabilidad horizontal.

2. Análisis del estándar JSON: ventajas sobre XML

Para la comunicación entre el cliente web de GoApple POS y el servidor, se utilizará HTTP con JSON (JavaScript Object Notation). Las ventajas competitivas de JSON sobre XML son:

Criterio	JSON	XML
Tamaño del Payload	Compacto y ligero	Verboso, mayor tamaño
Procesamiento	Nativo en JavaScript (sin librerías)	Requiere parser externo
Legibilidad	Alta para humanos y máquinas	Compleja con namespaces y atributos
Latencia HTTP	Mínima (payloads pequeños)	Mayor (overhead de etiquetas)

3. Servicios externos identificados

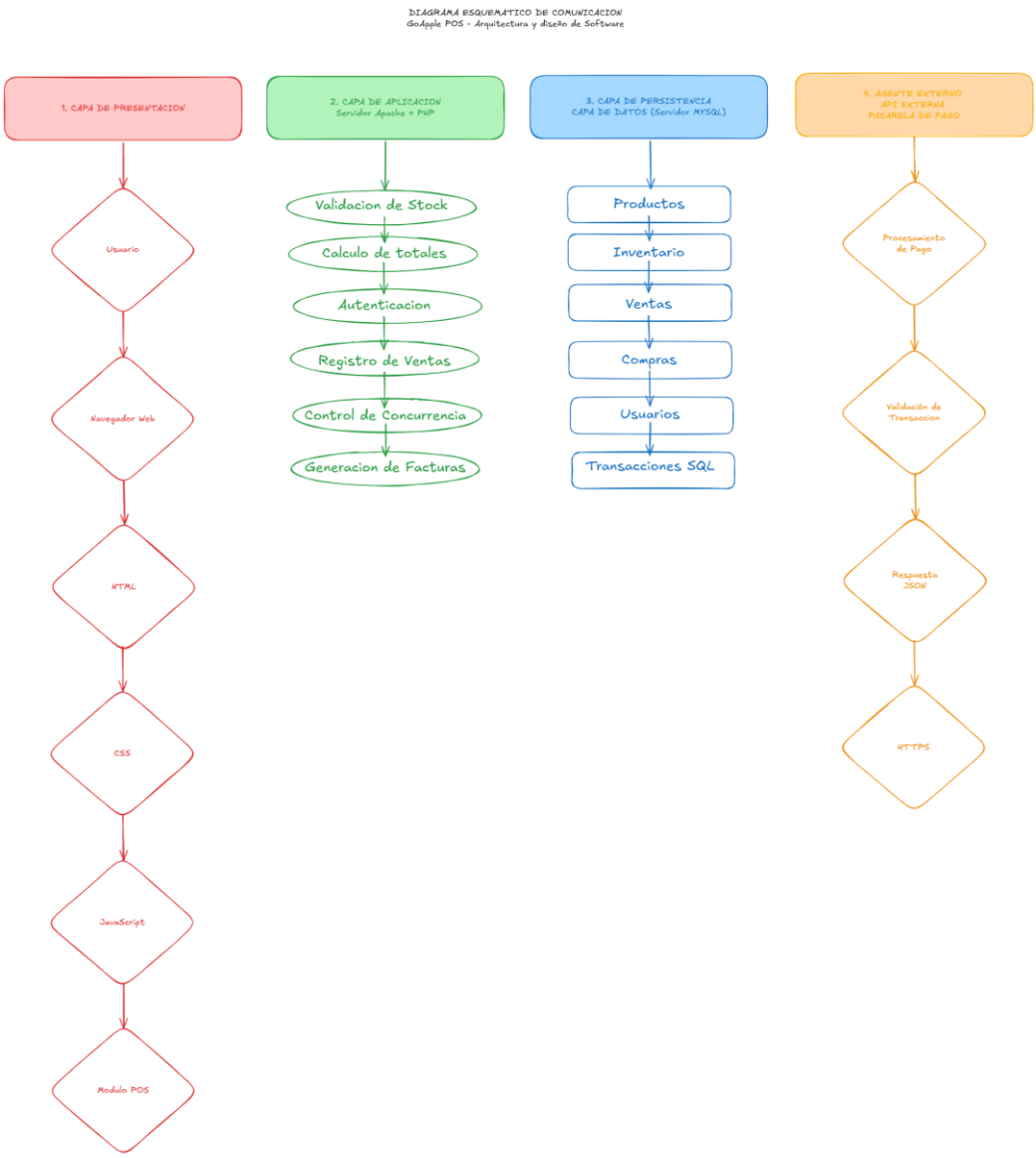
Dado que GoApple POS maneja registro de ventas y facturación, se identifican dos agentes externos:

- API de Pasarela de Pagos (Stripe o Mercado Pago): para procesar pagos electrónicos de manera segura.
- Motor MySQL como nodo externo de persistencia: almacena productos, ventas, compras y usuarios con integridad referencial.

Actividad 2: Modelado del Diagrama Esquemático de Comunicación

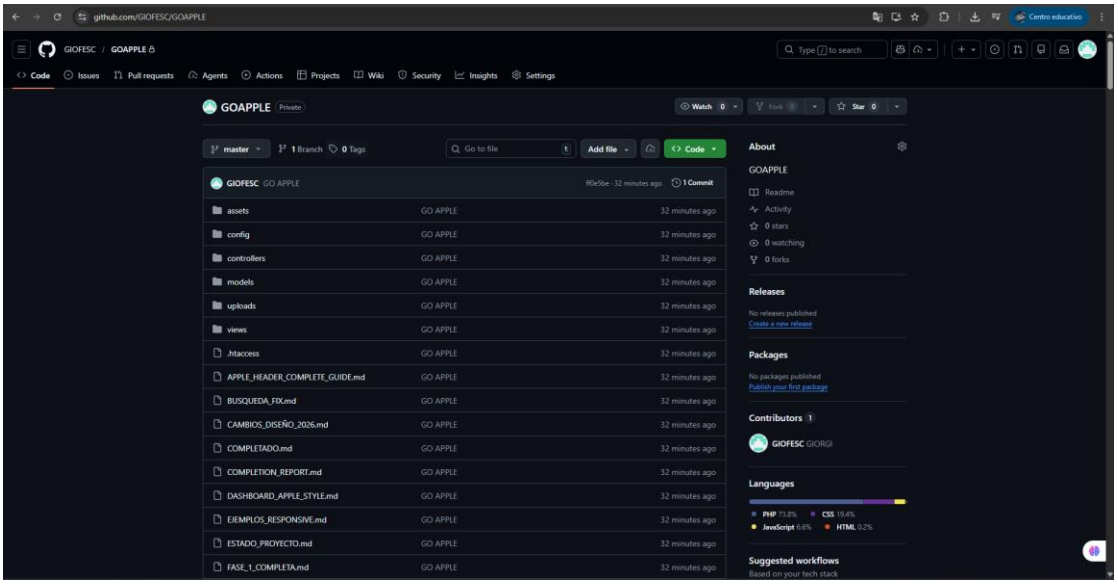
Diagrama elaborado con herramienta profesional (Draw.io). Representa la interacción técnica entre los 4 niveles del sistema:

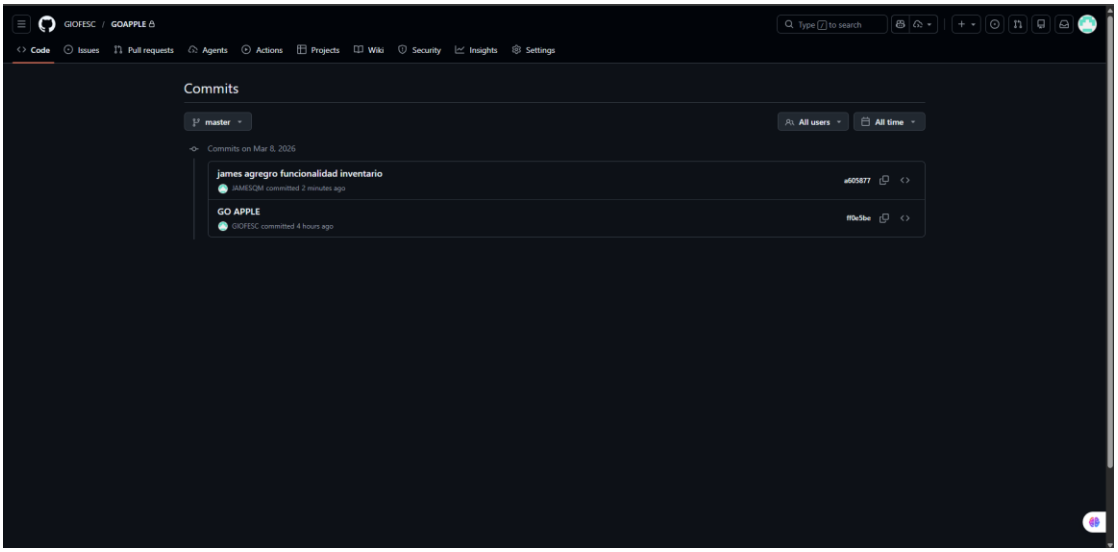
- Capa de Presentación: Interfaz de usuario / Navegador web.
- Capa de Aplicación: Lógica de negocio en el servidor PHP.
- Capa de Persistencia: Servidor de Base de Datos MySQL.
- Agentes Externos: API de Pasarela de Pagos.



Actividad 3: Gestión de Configuración y Versionamiento (Git/GitHub)

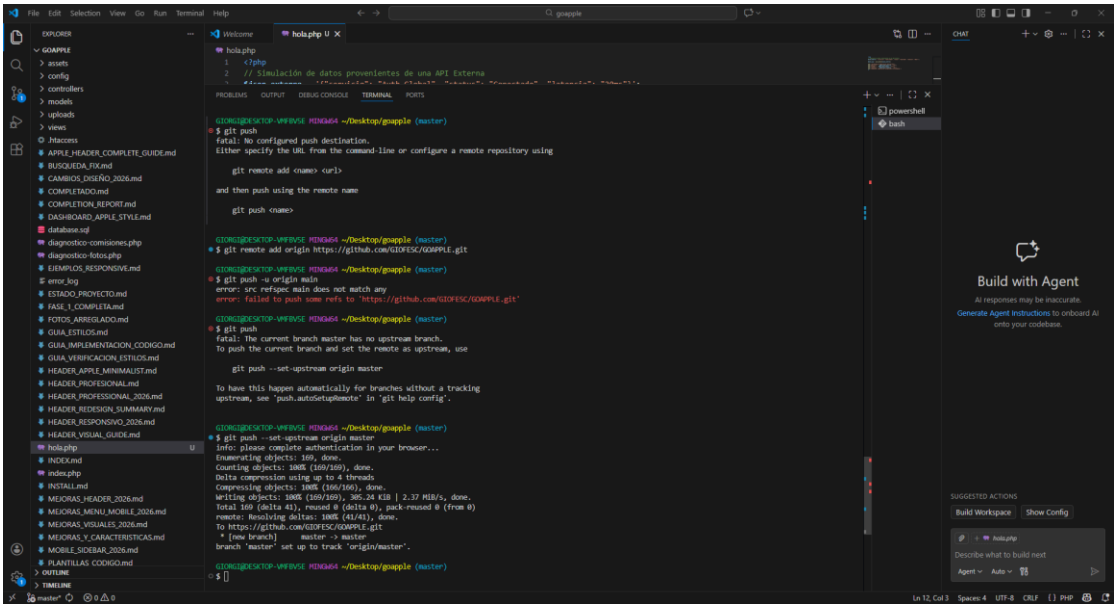
Se implementó el sistema de control de versiones Git y se vinculó el repositorio con GitHub para gestionar el código fuente del proyecto GoApple de manera colaborativa y con historial de cambios.





Actividad 4: Implementación Técnica — Simulación de Consumo de Datos

Se implementó la simulación de consumo de datos mediante fetch() y JSON, demostrando la comunicación entre la capa de presentación y el servidor PHP con el estándar de intercambio definido.


File Edit Selection View Go Run Terminal Help
goapple
EXPLORER
GOAPPLE
assets
config
controllers
models
uploads
views
.htaccess
APPLE_HEADER_COMPLETE_GUIDE.md
BUSQUEDA_FIX.md
CAMBIOS_DISEÑO_2026.md
COMPLETADO.md
COMPLETION_REPORT.md
DASHBOARD_APPLE_STYLE.md
database.sql
diagnostico-comisiones.php
diagnostico-fotos.php
EJEMPLOS_RESPONSIVE.md
error_log
ESTADO_PROYECTO.md
FASE_1_COMPLETA.md
FOTOS_ARREGLADO.md
GUIA_ESTILOS.md
GUIA_IMPLEMENTACION_CODIGO.md
GUIA_VERIFICACION_ESTILOS.md
HEADER_APPLE_MINIMALIST.md
HEADER_PROFESIONAL.md
HEADER_PROFESSIONAL_2026.md
HEADER_REDESIGN_SUMMARY.md
HEADER_RESPONSIVO_2026.md
HEADER_VISUAL_GUIDE.md
hola.php
INDEX.md
index.php
INSTALL.md
MEJORAS_HEADER_2026.md
MEJORAS_MENU_MOBILE_2026.md
MEJORAS_VISUALES_2026.md
MEJORAS_Y_CARACTERISTICAS.md
MOBILE_SIDEBAR_2026.md
PLANTILLAS_CODIGO.md
OUTLINE
TIMELINE
Welcome
hola.php U
hola.php
1 <?php
2 // Simulación de datos provenientes de una API Externa
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
GIORGI@DESKTOP-VMF8V5E MINGW64 ~/Desktop/goapple (master)
$ git push
fatal: No configured push destination.
Either specify the URL from the command-line or configure a remote repository using
git remote add <name> <url>
and then push using the remote name
git push <name>
GIORGI@DESKTOP-VMF8V5E MINGW64 ~/Desktop/goapple (master)
$ git remote add origin https://github.com/GIOFESC/GOAPPLE.git
GIORGI@DESKTOP-VMF8V5E MINGW64 ~/Desktop/goapple (master)
$ git push -u origin main
error: src refspec main does not match any
error: failed to push some refs to 'https://github.com/GIOFESC/GOAPPLE.git'
GIORGI@DESKTOP-VMF8V5E MINGW64 ~/Desktop/goapple (master)
$ git push
fatal: The current branch master has no upstream branch.
To push the current branch and set the remote as upstream, use
git push --set-upstream origin master
To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.
GIORGI@DESKTOP-VMF8V5E MINGW64 ~/Desktop/goapple (master)
$ git push --set-upstream origin master
info: please complete authentication in your browser...
Enumerating objects: 169, done.
Counting objects: 100% (169/169), done.
Delta compression using up to 4 threads
Compressing objects: 100% (166/166), done.
Writing objects: 100% (169/169), 305.24 KiB | 2.37 MiB/s, done.
Total 169 (delta 41), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (41/41), done.
To https://github.com/GIOFESC/GOAPPLE.git
* [new branch] master -> master
branch 'master' set up to track 'origin/master'.
GIORGI@DESKTOP-VMF8V5E MINGW64 ~/Desktop/goapple (master)
$
powershell
bash
CHAT
Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI onto your codebase.
SUGGESTED ACTIONS
Build Workspace
Show Config
hola.php
Describe what to build next
Agent
Auto
Ln 12, Col 3
Spaces: 4
UTF-8
CRLF
PHP

GUÍA N°4 ESPECIFICACIÓN DE ATRIBUTOS DE CALIDAD Y MÉTRICAS PARA GOAPPLE

Fecha	02 de marzo de 2026
Autores	James Quintero Mosquera Giorgi Julián Ordóñez

Actividad 1: Matriz de Atributos de Calidad

Basado en los modelos de calidad de Sommerville y Pressman, se formaliza la siguiente matriz técnica con los umbrales de cumplimiento para GoApple:

Atributo	Métrica / Umbral	Estrategia de Implementación	Herramienta / Técnica
Rendimiento	Tiempo de respuesta ≤ 1.5 s (página completa en localhost)	Índices en columnas FK. Consultas SQL optimizadas con SELECT específico. Caché de sesión PHP.	índices MySQL
Seguridad	0 vulnerabilidades XSS/SQL Injection en formularios críticos	Validación con filter_var(). Prepared Statements con PDO. Sanitización con htmlspecialchars().	PHP PDO, htmlspecialchars(), OWASP Top 10
Confiabilidad	Disponibilidad ≥ 99 % Tiempo medio entre fallos > 30 días	Manejo de excepciones try/catch. Logs de error en /logs/. Transacciones MySQL (BEGIN/COMMIT).	error_log(), PDO::beginTransaction(), XAMPP
Usabilidad	≤ 3 clics para completar una venta o registro	Interfaz responsiva con CSS Grid/Flexbox. Mensajes de error claros. Validación en tiempo real con JS.	CSS nativo, JavaScript validación
Mantenibilidad	Cobertura de comentarios ≥ 70 % en clases del modelo	Separación MVC estricta. Nomenclatura PSR-12. Documentación PHPDoc en modelos.	PSR-12, PHPDoc, /models /views /controllers
Escalabilidad	Soporte ≥ 10 usuarios concurrentes sin degradación	Arquitectura stateless en PHP. Separación BD / lógica. Posibilidad de migrar a servidor VPS externo.	Apache mod_php, MySQL connection pooling

1.1 Rendimiento

GoApple opera en un entorno PHP/XAMPP donde el tiempo de respuesta es crítico para la experiencia del cajero. Se establecen las siguientes métricas:

- Carga inicial del dashboard: ≤ 1.5 segundos en localhost.
- Registro de una venta completa: ≤ 2 segundos.

- Consulta de inventario (100–500 productos): ≤ 0.8 segundos.

Estrategias: opcache PHP para caché de bytecode, índices en columnas de búsqueda frecuente, consultas SELECT con campos específicos en lugar de SELECT *.

1.2 Seguridad

- Validación de entradas con filter_var() y filter_input() en todas las rutas del servidor.
- Prevención de SQL Injection mediante Prepared Statements con PDO.
- Sanitización con htmlspecialchars() para prevenir XSS en vistas.
- Control de acceso por roles: administrador y vendedor con sesiones PHP verificadas en cada request.

1.3 Confiabilidad y Usabilidad

Confiabilidad — Mecanismos de tolerancia a fallos:

- Manejo de excepciones con bloques try/catch en todas las operaciones de base de datos.
- Transacciones MySQL (BEGIN / COMMIT / ROLLBACK) en operaciones multi-tabla.
- Sistema de logs en /logs/error.log con registro de errores críticos mediante error_log().

Usabilidad — Criterios de experiencia de usuario:

- Navegación en máximo 3 clics desde el dashboard hasta completar cualquier operación.
- Mensajes de retroalimentación claros en cada acción (éxito, error, advertencia).
- Interfaz responsiva en tablets y PC de escritorio.
- Validación en tiempo real con JavaScript antes del envío al servidor.

Actividad 2: Análisis Prospectivo — El "Costo de Cambio"

La arquitectura MVC de GoApple reduce significativamente el impacto técnico y económico ante cambios mayores. Se evalúan los escenarios de modificación más probables:

Escenario de Cambio	Capa Afectada	Impacto sin MVC	Impacto con MVC	Estimado Horas
Migración MySQL → PostgreSQL	Capa de Datos	Alto: reescribir toda la lógica de consultas	Bajo: solo modificar database.php y modelos	4–8 h
Cambio Apache → Nginx	Infraestructura	Medio: revisar rutas .htaccess	Mínimo: ajustar config del servidor	1–2 h

Escenario de Cambio	Capa Afectada	Impacto sin MVC	Impacto con MVC	Estimado Horas
Módulo facturación electrónica	Capa de Negocio	Alto: acoplar con código existente	Bajo: nuevo controlador + modelo	6–12 h
Rediseño completo del frontend	Capa de Presentación	Alto: cambios en cada script PHP	Mínimo: solo archivos /views/	3–6 h
Autenticación OAuth 2.0	Negocio + Config	Alto: cambios transversales	Bajo: middleware en /Core/	4–8 h

2.1 Escenario Principal: Migración de MySQL a PostgreSQL

Con la arquitectura MVC de GoApple, el cambio se limita a:

- Modificar database.php en /config/ para cambiar el DSN de mysql: a pgsql:.
- Adaptar los modelos en /models/ que usan sintaxis específica de MySQL (ej: AUTO_INCREMENT → SERIAL).
- La capa de presentación (/views/) y los controladores (/controllers/) permanecen sin modificación.

Estimación de impacto económico: sistema monolítico ~40–80 horas; arquitectura MVC de GoApple ~4–8 horas. Reducción del costo de cambio: 80–90%.

Actividad 3: Implementación de Capa de Validación (Validator.php)

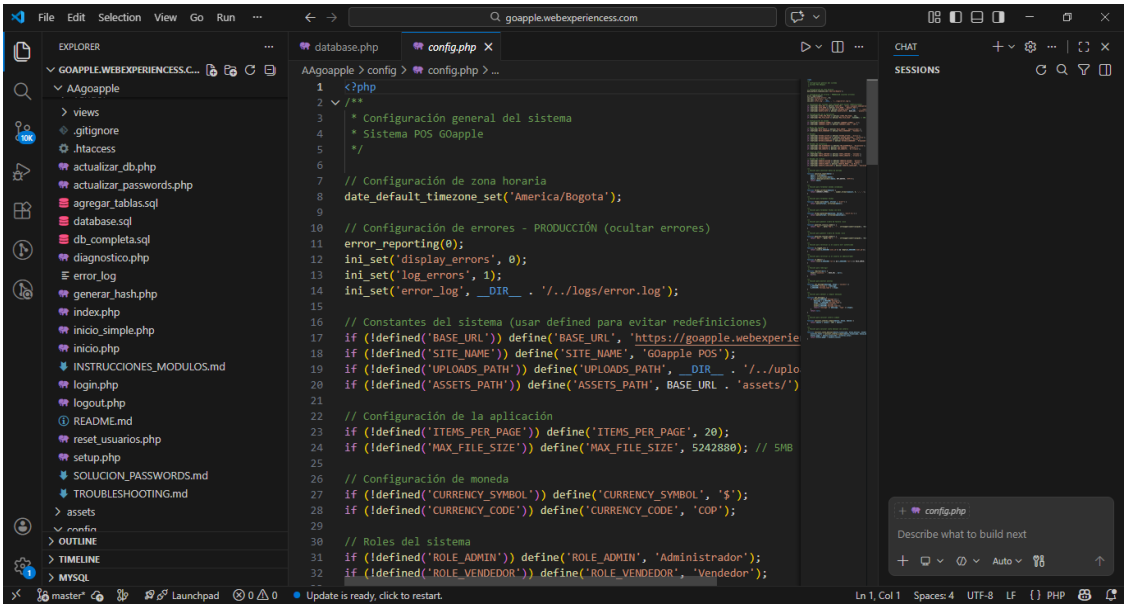
Se desarrolla el componente Validator.php dentro del directorio /app/Core/ del proyecto GoApple. Este componente intercepta los datos de entrada antes de su procesamiento en la capa de negocio, aplicando filtros nativos de PHP.

3.1 Ubicación en la Arquitectura

Ruta: /goapple/app/Core/Validator.php

- Actúa como middleware entre el request HTTP y la capa de negocio.
- Es invocado desde los controladores antes de pasar datos a los modelos.
- Centraliza toda la lógica de validación evitando duplicación de código.

3.2 Código Fuente — Evidencia en VS Code



3.3 Métodos del Componente

Método	Filtro PHP	Descripción
texto()	FILTER_SANITIZE_SPECIAL_CHARS	Valida nombres, descripciones y campos de texto libre.
entero()	FILTER_VALIDATE_INT	Valida cantidades, stock, IDs numéricos.
decimal()	FILTER_VALIDATE_FLOAT	Valida precios, montos de ventas y créditos.
email()	FILTER_VALIDATE_EMAIL	Valida correos de clientes y usuarios del sistema.
telefono()	preg_match — regex	Valida celulares colombianos (10 dígitos, inicio 3).
tipoVenta()	in_array() estricto	Restringe el tipo de venta a valores permitidos.
sinInyeccion()	stripos() — lista negra	Detecta patrones peligrosos de SQL Injection.

Actividad 4: Consolidación del Dossier Arquitectónico

Se organiza de forma sistemática toda la documentación técnica generada durante el semestre. Este dossier constituye el portafolio técnico completo del proyecto GoApple y será el insumo principal para la sustentación presencial.

4.1 Índice del Portafolio Técnico

N°	Sección	Descripción	Archivo
1	Guía Práctica N°1	Planteamiento del proyecto, arquitectura conceptual y física, patrones MVC	Guia_N1_GoApple.docx

N°	Sección	Descripción	Archivo
2	Guía Práctica N°2	Entrenamiento PHP, retos de código, frameworks, arquitectura en VS Code	Guia_N2_GoApple.docx
3	Guía Práctica N°3	APIs, JSON vs XML, diagrama de comunicación, Git/GitHub, simulación datos	Guia_N3_GoApple.docx
4	Guía Práctica N°4	Matriz calidad, costo de cambio, Validator.php, dossier técnico	Guia_N4_GoApple.docx
5	Diagrama de Comunicación	Representación visual 4 capas: Presentación → Negocio → Datos → APIs	Diagrama_Comunicacion.png
6	Estructura de Directorios	Árbol físico del proyecto GoApple con responsabilidades por capa	Estructura_Fisica.png
7	Código — Validator.php	Componente de validación en /app/Core/. Capturas en VS Code	Validator_VSCode.png
8	Base de Datos	Script SQL: productos, clientes, ventas, detalle_venta, usuarios, creditos	database.sql

Nota Final — Preparación para la Sustentación

Este documento unificado completa el dossier arquitectónico del proyecto GoApple para la asignatura Arquitectura y Diseño de Software (FESC). Para la sustentación presencial se debe disponer de: (1) el proyecto funcional en XAMPP/Laragon con la estructura MVC implementada, (2) este portafolio en formato digital o impreso, (3) el diagrama de comunicación de la Guía N°3, y (4) demostración del componente Validator.php en ejecución.